

Supplemental Material to “Exact Solution to Moving Horizon Estimation: Theory and Benchmark Application”

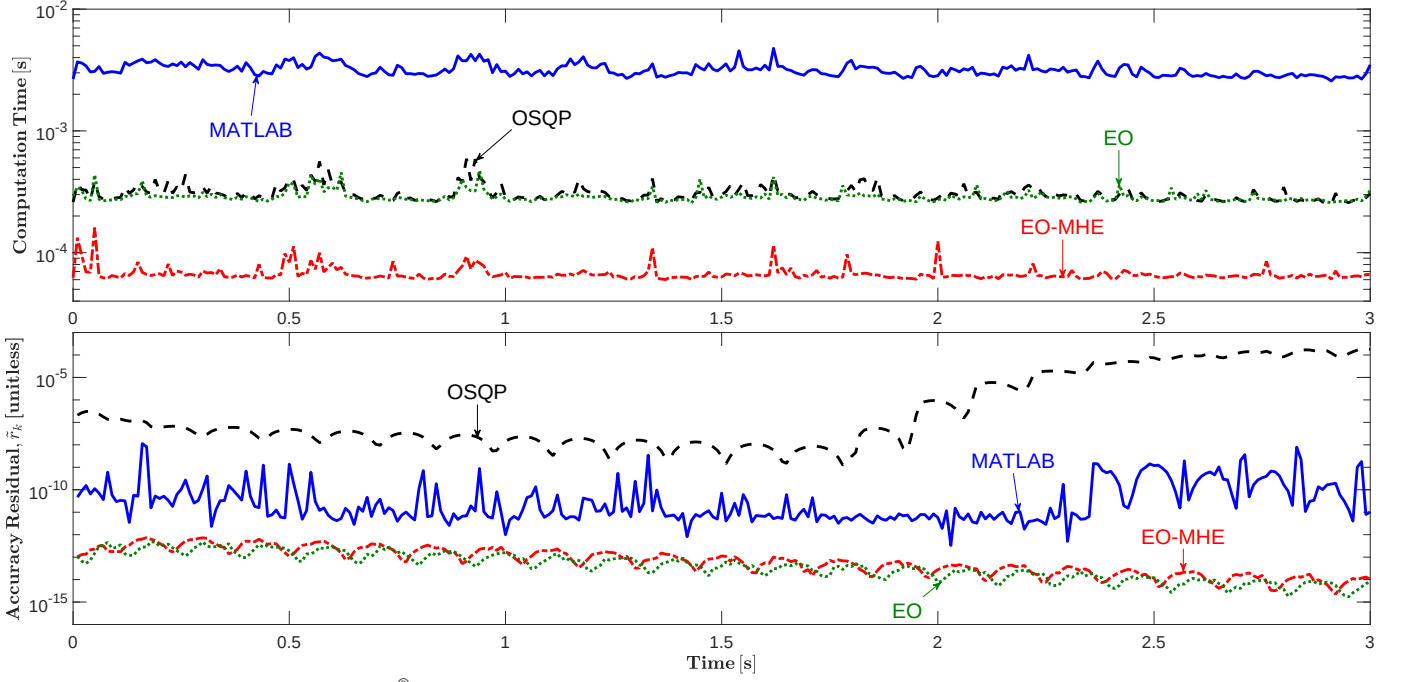


Fig. 3. (Numerical Validations on MATLAB® for Suspension System in Eq. (10)) Time histories of the computation time and accuracy of the MHE solvers: MATLAB® (“quadprog” function), OSQP [18], EO [21], and the proposed EO-MHE scheme, where the accuracy is in terms of the residual $\tilde{r}_k$ in Eq. (13) and $N = 10$ [10] ($n_D = 260$).

APPENDIX A

REMARKS OF SEC. IV-A (REMARKS 5-7)

Remark 5: This remark further clarifies the fairness of computational comparison; more specifically, it exemplifies 2) of Remark 2 using the benchmark application in Sec. IV. With respect to each method in Fig. 3, most matrix-related computations can be computed offline, including the input-related parameter $\tilde{\mathbf{b}} = \mathbf{0}$ that results from the considered, passive Suspension System [26]; whereas those computations associated with the varying output measurements $\mathbf{q}$ should be computed online, from one estimation cycle to the next one. Accordingly, the computation time in Fig. 3 (a) accounts for the online computations only (q-related updates) for all the illustrated MHE solvers, which (arrangement) aims at a fair computational comparison.

Remark 6: Moving forward from the validations in Sec. IV-A, this remark further evaluates how the EO-MHE’s performances scale with N and n , respectively, as well as the associated n_D .

- N : To be more focused on the N effect, the following notations and parameter settings are identical to or similarly extend from those in Sec. IV-A (for example, $n = 4$), but not detailed for brevity. We continue the evaluations in Table I and enlarge the horizon length to $N = 30$. The problem dimension goes to $n_D = 760$; the computation times [s] are 3.3×10^{-3} using “quadprog” in MATLAB®, 3.9×10^{-4} (OSQP), 1.5×10^{-3} (EO), and 1.7×10^{-4} (EO-MHE); and the accuracy residuals [unitless] are 4.3×10^{-9} (MATLAB®), 6.8×10^{-4} (OSQP), 9.5×10^{-15} (EO), and 6.8×10^{-16} (EO-MHE). All these

numerics still favor the EO-MHE scheme, which is consistent with Table I. Similar advantages extend to larger-dimensional problems; nonetheless, we only, further, and similarly summarize the next case for conciseness and memory considerations in FPGA implementation. $n = 40$; $n_D = 1010$; times: 4×10^{-3} (MATLAB®), 4.7×10^{-4} (OSQP), 2.3×10^{-3} (EO), and 2.4×10^{-4} (EO-MHE); and residuals: 5.5×10^{-9} (MATLAB®), 1.2×10^{-3} (OSQP), 10^{-14} (EO), and 7.8×10^{-16} (EO-MHE). Remarkably, the additional evaluations ($n_D = 760$ and 1010) show that EO-MHE achieves even smaller residuals than all the other solvers (10^{-16} to the order, exclusively within Table I and above), while it completes the solving process faster than the others (around two times faster, at least).

- n : Similar to the above settings for N , now we turn to the scaling effect by the number of states. Following the preliminary quarter-car Suspension System in Eq. (10), we move on to its half-car extension for a more consistent and meaningful evaluation, which doubles the system states to $n = 8$ [27], [29]. Accordingly, the evaluation results are: $N = 10$; $n_D = 1144$; $0.02 \leq \max_k |e_k^i| \leq 1.3$ for each system state (denoted by the superscript i) and among the same four solvers, where the corresponding ARMS values are within the range $[8 \times 10^{-3}, 0.5]$ while the trajectories by the solvers still largely overlap as in Fig. 1; and computation times [s] as per Table I: 4.7×10^{-3} (MATLAB®), 4.8×10^{-4} (OSQP), 9.5×10^{-4} (EO), and 1.2×10^{-4} (EO-MHE), where the residuals [unitless] are 1.8×10^{-8} , 1.5×10^{-8} , 1.7×10^{-11} , and 2.2×10^{-13} , respectively. To summarize the above results,

EO-MHE still preserves estimation performance while improving computation time and solving accuracy than the baseline solvers, which (observations) comply with the lower-dimensional subsystem in Eq. (10).

Remarkably, the MHE formulation increases the memory demand from, for instance, $\mathbf{x}_k \in \mathbb{R}^n$ in Eq. (1) to $\tilde{\mathbf{x}}_k \in \mathbb{R}^{n(N+1)}$ in Eq. (4); while Corollary 1 analyzes that the computational complexity of the EO-MHE solver scales with $O(n^2 N^2)$. Regarding a high-dimensional or fast-dynamic system, the corresponding computational, memory overhead, or other hardware constraints may delimit the practical versatility of the proposed, theoretically general MHE solution. Along this research direction, the above evaluations in this Remark 6 still favor EO-MHE as n doubles (resp., N grows to 30); while Appendix B later demonstrates the generality subject to even larger values of N for faster-dynamic MHE designs.

Remark 7: In addition to the baseline comparisons so far, this remark presents validations and evaluations that are against other benchmark MHE solvers and in terms of demonstrations of EO-MHE's (i) application spectrum and (ii) integration of equality and inequality constraints.

- 1) Among various MHE implementations, a fast and embedded-oriented scheme has been widely adopted for control practice, namely [10, Sec. III-C], which is denoted "Exact KKT", can be traced back to its theoretical fundamentals [12, 2nd ed., Ch. 14], and still manifests popularity in the recent literature [6]. The motivations for such an exact solver highlight its remarkable efficiency and realization simplicity, which are particularly crucial for sensorless control [6], [10]; moreover, in the scope of Linear Time-Invariant systems, the computational speed can be further improved because the inverse of the underlying KKT matrix can be pre-computed offline. Accordingly, the Exact KKT solver serves as another reasonable benchmark to compare with EO-MHE. Under the same settings as in Sec. IV (for instance, $N = 10$ [10]), the comparison results are: $0.06 \leq \max_k |e_k^i| \leq 4.1$ for each system state and among the two solvers, where the corresponding ARMS values reside in $[10^{-2}, 1]$ while the responses by the solvers still largely overlap as in Fig. 1; and computation times [s] are 5.2×10^{-5} for Exact KKT and 4.3×10^{-5} for EO-MHE, where the residuals [unitless] are 4.4×10^{-7} and 1.3×10^{-13} , respectively. To sum up, EO-MHE still preserves estimation performance in a manner that is more computationally efficient than Exact KKT. Remarkably, EO-MHE exhibits analytical advantages over Exact KKT, such as: 1) the latter hinges upon the Lagrange necessary conditions [12] and thus requires further effort toward equivalent results; 2) the inverse computation in the latter only applies to the case of " $P \succ 0$ ", which incurs the common compromise that resorts to regularization techniques [10], [16], [20] (the popular Tikhonov regularization [10, Eq. (11)] is adopted in this comparison); 3) the regularization technique causes the Exact KKT scheme to suffer from the tradeoff tuning between ill-conditioned computation and larger estimation errors, which arises from the ad-

ditional perturbation parameter; and 4) the speedup by pre-computing the inverse in Exact KKT is not general across MHE problems, to exemplify using the same recent literature, estimation in a motor control system [6], where EO-MHE can still be **applied**.

- 2) Continuing Item 1), this remark considers another fast, embedded-oriented MHE implementation [6, Sec. III-B] in the recent literature. The major difference compared with Exact KKT [10] is the underlying regularization technique; nonetheless, the evaluations also manifest similar performance advantages using EO-MHE, and thus the details are not repeated for compactness.
- 3) Speaking of another case study [6], it is worth evaluating the capabilities and potential of EO-MHE by (a) further comparing with standard Riccati-based solvers [30] and (b) demonstrating how the method scales to handle inequality constraints; however, the results are summarized in Appendix B below to maintain the focus of this brief.

APPENDIX B

CASE STUDY OF ESTIMATION IN MAGLEV TRAIN [6]

In a similar manner to Sec. IV, we consider the estimation problem of a motor control system in a maglev train according to [6], which conforms to Eq. (1), where

$$A = \begin{bmatrix} 1 - T_s R_s / L_d & a_{12} & T_s / L_d & 0 \\ -a_{12} & 1 - T_s R_s / L_d & 0 & -T_s / L_d \\ 0 & 0 & 1 & \omega_e T_s \\ 0 & 0 & -\omega_e T_s & 1 \end{bmatrix},$$

$$C = \begin{bmatrix} \mathbf{I}_2 & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{2 \times 4},$$

$T_s = 10^{-4}$ [s], $R_s = 0.3$ [Ω] the phase resistance, $L_d = 3.9$ (resp., $L_q = 3.6$) [mH] the d (resp., q)-axis inductance in the rotating frame, $a_{12} = \omega_e (L_q - L_d) T_s / L_d$, $\omega_e := \omega_e(t) \in \mathbb{R}_{\geq 0}$ [rad/s] the electrical angular velocity; $\mathbf{x}_k = [x_k^i]_{i=1}^4 \in \mathbb{R}^4$ that includes currents (x_k^1 and x_k^2) and components of extended electromotive forces (x_k^3 and x_k^4) with respect to the stationary reference frame; $\xi_k = \eta_k = \mathbf{0}$; and more details are provided in [6] for brevity. Note that the time dependence of ω_e renders the estimation system time-variant, which clarifies why pre-computing the inverse in the Exact KKT scheme cannot be applied in this estimation study, as discussed in 1) of Remark 7 in Appendix A. The objective is to estimate $\{x_k^i\}_{i=1}^4$, where $\{x_k^j\}_{j=1,2}$ are prone to measurement noise [6]; while the focus is on the performance comparison between EO-MHE and standard Riccati-based solvers, demonstratively, Fixed-Lag Smoother (FLS) and Information Filter (IF) [30, Chs. 6 and 9]. Given $n = 4$ and $m = p = q = 2$, the validation settings for Eqs. (2)-(4) start with $R = I_2$, $\Pi = I_4$, $N = 3$, and $\mathbf{x}_k = \hat{\mathbf{x}}_{k|k} = \mathbf{0}$, where $k = 0, 1, 2, 3$ [6]; while the initial covariance matrix using FLS (resp., IF) is $\text{diag}([10^{-4}, 10^{-4}, 0.3, 0.3])$. Accordingly, (A, C) is observable; while P in Eq. (4) is singular, which also exemplifies 4) and 5) of Remark 2 with $\det(V_2^T P V_2) = 389 \neq 0$. Lastly, we consistently adopt the metric in Eq. (12) to evaluate the estimation errors.

Similar to Fig. 1 (resp., [24, Fig. 3]), Fig. 4 (resp., 5) shows the MATLAB® responses and compares the four methods:

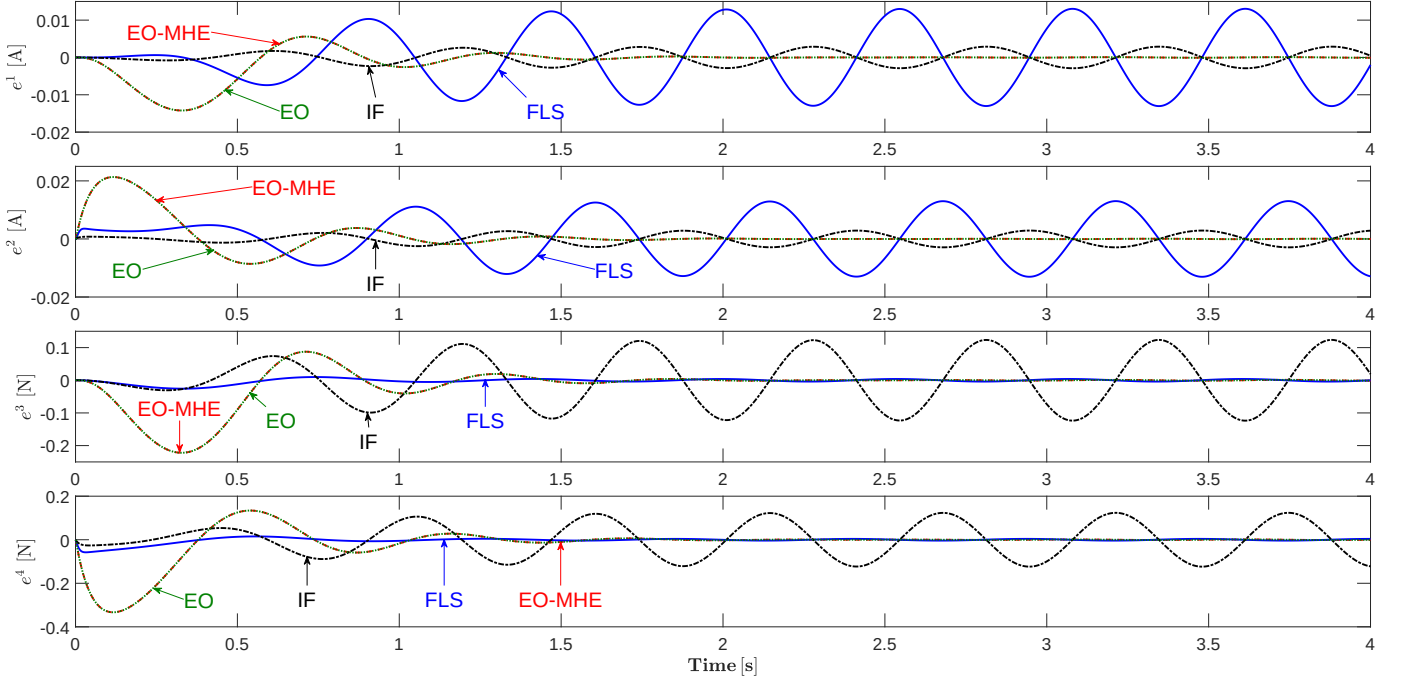


Fig. 4. (Numerical Validations on MATLAB[®] for Appendix B) Time histories of the estimation errors $\{e_k^i\}_{i=1}^4$ in Eq. (12), where four methods are compared: FLS [30], IF [30], EO [21], and the proposed EO-MHE scheme – the latter two are associated with $N = 3$ [6] ($n_D = 52$).

- Riccati-based solvers [30]: FLS and IF using the blue/solid and black/dash-dotted lines, respectively;
- EO [21] and its tailored upgrade “EO-MHE” by the green/dotted and red/dashed lines, respectively.

Fig. 4 plots the estimation error in terms of Eq. (12) and suggests that only the behaviors of EO and EO-MHE are converging to zero. More specifically, the green and red trajectories largely overlap and almost approach zero within four seconds; whereas both the blue (resp., black) lines in the upper (resp., lower) two subfigures oscillate throughout most of the time horizon, particularly at the end. To be more detailed about the **improved** evaluations, $0.01 \leq \max_k |e_k^i| \leq 0.3$ for each system state and ARMS values are within $[7 \times 10^{-3}, 0.1]$ using both the EO and EO-MHE schemes. Actually, the two schemes are not only advantageous in terms of the metric (12), but also the solving time and accuracy according to Fig. 5, which averages 10^2 random runs at each instant. Upon examining Fig. 5, it is evident that the green (resp., red) trajectory does not intersect with the others in the upper subfigure, while EO-MHE still requires the shortest time as in the benchmark study ([24, Fig. 3]). Quantitatively speaking, the average computation time for the red response is 1.4×10^{-5} [s], which is at least 2 times faster than EO, 3 for FLS, and 4 for IF. Such an observation also reiterates 2) of Remark 1 that motivates this study. Moving forward to the other accuracy metric in Eq. (13), the lower subfigure of Fig. 5 further endorses the solving accuracy using EO and EO-MHE. Notably, the average residual of EO-MHE is around 2.8×10^{-15} [unitless], which still consistently approaches the default precision setting in MATLAB[®]. Note that the residual subfigure excludes FLS and IF, because such estimation designs do not adopt the evaluation metric (13). Moreover, the remaining materials provide

TABLE II
(NUMERICAL VALIDATIONS ON MATLAB[®] FOR APPENDIX B) AVERAGE COMPUTATION TIME OF 10 RUNS SUBJECT TO VARIOUS HORIZON LENGTHS (N), PROBLEM DIMENSIONS (n_D USING EQ. (14)), AND THE FOUR METHODS IN FIG. 5.

N	n_D	Computation Time [s]			
		FLS	IF	EO	EO-MHE
3	52	5.3×10^{-5}	5.6×10^{-5}	3×10^{-5}	1.5×10^{-5}
10	164	1.3×10^{-4}	1.5×10^{-4}	1.8×10^{-4}	3.7×10^{-5}
20	324	2.7×10^{-4}	3×10^{-4}	6.3×10^{-4}	8.6×10^{-5}
50	804	6.3×10^{-4}	6.8×10^{-4}	3.5×10^{-3}	2.6×10^{-4}
89	1428	1.2×10^{-3}	1.2×10^{-3}	1.2×10^{-2}	9.2×10^{-4}

scalability evaluations with regard to problem dimensions and inequality constraints, respectively.

On the one hand, Table II evaluates the scalability of the proposed scheme, compares the results with the other methods in Fig. 5, and adopts similar settings as in Table I for consistency, notably, averaging 10 random runs and categorizing the problem dimension as per Eq. (14) [18]. Regarding the first three horizons ($N = 3, 10$, and 20 ; while $n_D = 52, 164$, and 324), EO-MHE still yields the shortest computation time; in other words, when the horizon length is small, EO-MHE entails fast computation for estimation. The observations are consistent with analytical results; more specifically, Corollary 1 endorses that the EO-MHE method scales with $O(N^2)$, which enriches the diversity of efficient solvers that are typically recursive/Riccati-based with $O(N)$ [30]. Although practical considerations for real-time implementations favor small horizons and dimensions (such as [10] and up until

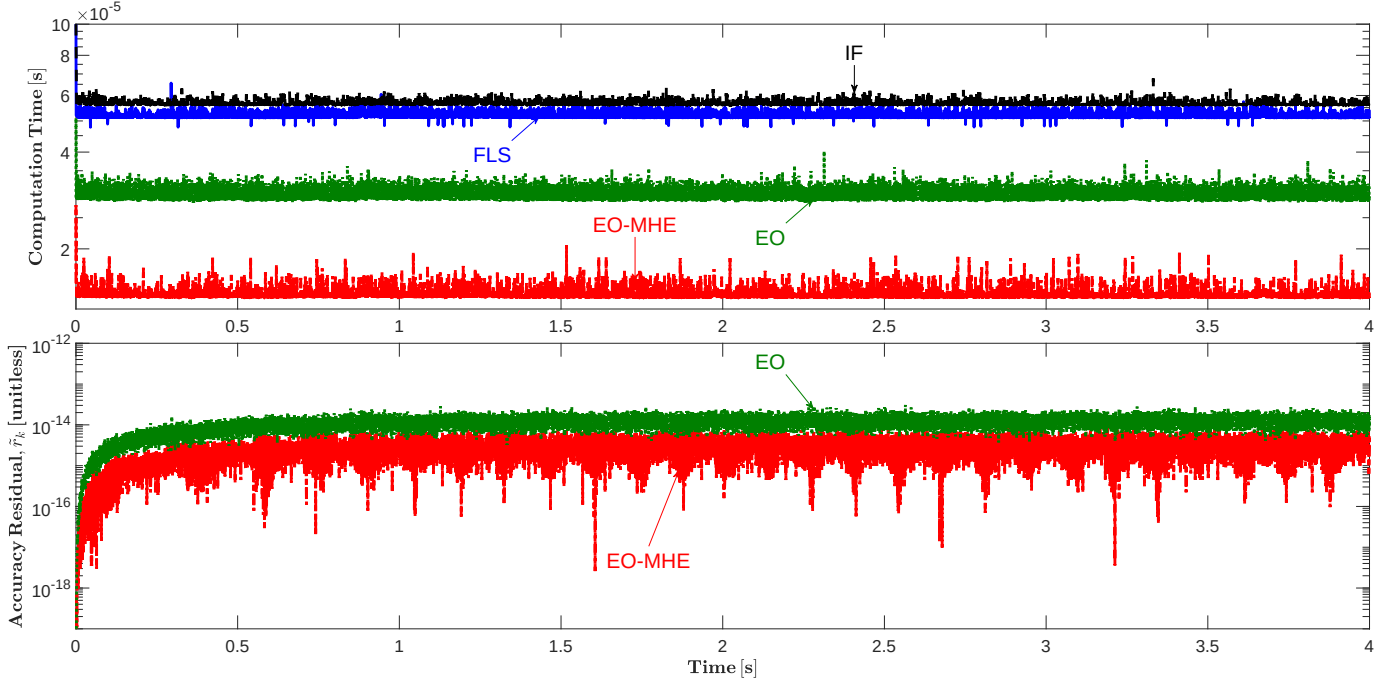


Fig. 5. (Numerical Validations on MATLAB® for Appendix B) Time histories of the average computation time and accuracy residual over 10^2 random runs using the four estimation methods: FLS [30], IF [30], EO [21], and the proposed EO-MHE scheme, where the residual is in terms of Eq. (13) and $N = 3$ [6] ($n_D = 52$) for EO and EO-MHE.

the latest literature [6]), Table II further provides two more evaluations. The first one increases to $N = 50$ ($n_D = 804$), where EO-MHE still marks the fastest result (at least 2.4 times faster than the others); while the other ($n_D = 1428$) reaches the scalability margin, beyond which the scheme requires more time than the baseline solvers. Remarkably, it is worth mentioning that the computation times using EO-MHE slightly vary between 10^{-5} and 10^{-4} [s], to the order; whereas those by EO are much more significant, which increase from 10^{-5} to 10^{-2} [s]. Such a customized, computational enhancement from EO to EO-MHE also exemplifies a motivation in 2) of Remark 1.

On the other hand, Fig. 6 continues Figs. 4 and 5, and it depicts the estimation results that further take physical bounds on states into consideration; analytically speaking, Fig. 6 evaluates how EO-MHE scales to explicitly handles inequality constraints. Such additional evaluations respond to one of the major motivations for using MHE over classic Kalman/Riccati solvers [30], and thus Fig. 6 excludes the results of FLS and IF as in Figs. 4 and 5. The physical bounds are imposed on the currents “ $x_k^1 = i_\alpha$ and $x_k^2 = i_\beta$ ” [A] according to the hardware limits of the experimental platform [6], specifically, $|x_k^i| < 39$ [A], $i = 1, 2$. The corresponding MHE formulation generalizes from ECQP-MHE in Eq. (4), by repeating “ $N + 1 = 4$ ” times for the inequalities (since $\mathbf{x}_k$ is repeated “ $N + 1$ ” times in the $\tilde{\mathbf{x}}_k$ formulation); accordingly, there are “ $n(N + 1) = 16$ ” inequality constraints in the estimation/optimization problem, which is denoted by QP-MHE. Along with this integration of inequalities, other problem settings are similarly adopted from Figs. 4 and 5 for a consistent evaluation of scalability, such as $n_D = 52$ and the residual metric in Eq. (13); while the practically direct extension to the QP solver [21,

Alg. 3.1] is invoked. In the upper subfigures of Fig. 6, the estimation results for the system states i_α and i_β are compared with the real values, all of which are within the physical bounds [6] and can barely be distinguished. Not only the satisfactory, negligible estimation errors but also the computational efficiency is highlighted, which is endorsed by the bottom subfigure of Fig. 6. Following Fig. 5 and averaging the results over 10^2 random runs, the computation time using EO-MHE (resp., EO) is 1.2×10^{-4} (resp., 1.4×10^{-4}) [s]; while the accuracy residual is 2.8×10^{-15} (resp., 1.1×10^{-14}) [unitless], which still approaches the default precision setting as in Fig. 5. It is worth mentioning that (1) the performance advantages of EO-MHE over EO in terms of solving time and accuracy also exemplify 2) of Remark 1; and (2) the scaling extension from ECQP-MHE to QP-MHE merely increases the computation time of the proposed EO-MHE solver by a factor of 8, which is much lower than a complexity estimation that amounts to “ $2^{16} = 65536$ ” times longer. The significant growth by the estimation is mainly due to the conditional, for-all environment [21, lines 12-29 of Alg. 3.1]; however, all the 4×10^6 exact-solving tasks in Fig. 6 complete at line 6 of the algorithm, which dominantly explains the computational efficiency, enriches practical capabilities and potential of EO-MHE, and encourages further research into the case of active inequality constraints.

APPENDIX C

REMARKS OF SEC. IV-B

We further discuss the practical merits (in addition to those in the beginning of Sec. IV-B) of the FPGA implementation of EO-MHE by, for example, comparing with the other MHE solvers in Fig. 3 that cannot be included in Fig. 2.

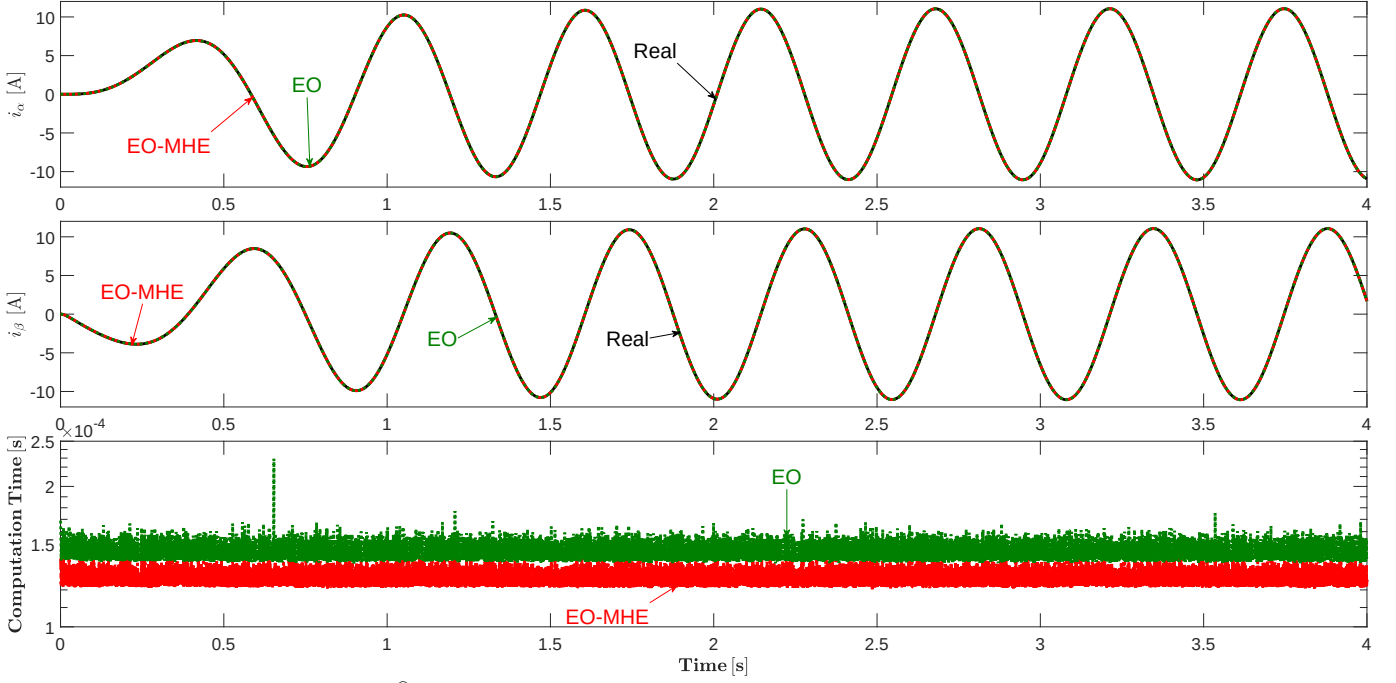


Fig. 6. (Numerical Validations on MATLAB® for Appendix B) Time histories of the constrained system states (i_α and i_β currents) and average computation time over 10^2 random runs, where the estimations by EO [21] and the proposed EO-MHE are compared with the real states; $N = 3$ [6] ($n_D = 52$); and the corresponding MHE formulation additionally integrates 16 inequality constraints.

- Numerical solvers [17] such as MATLAB® and OSQP in Fig. 3 entail intermediate-variable storage, convergence-monitoring mechanisms, and an indeterminate number of matrix-vector operations; whereas the exact solver (EO-MHE) can be evaluated using a predefined arithmetic sequence. Moreover, owing to the fundamental difference in methodology, it requires extra effort to filter out the factor associated with the architectural design of FPGA hardware, which also amounts to difference in computation time but is quite different between the numerical and exact solvers; therefore, these observations also help explain why Fig. 2 excludes numerical solvers.
- The non-iterative nature of EO-MHE avoids any convergence-dependent or residual-based stopping criteria, as well as any recursive updates in an online computational routine. Moreover, the exact nature allows a fixed system configuration that facilitates 1) offline computation of constant, matrix-related expressions and thus 2) reuse of the precomputed values at successive estimation cycles; while the remaining online computation can be mapped onto structure-fixed allocation of floating-point arithmetic units and predictable memory-access patterns. Note that, since the required arithmetic sequence is known a priori, a fixed-latency implementation renders a predetermined execution schedule.
- Considering potential gains from hardware optimizations, we demonstrate the use of pipelining for EO-MHE. Pipeline registers can be introduced between arithmetic stages to shorten critical combinational paths, which enables a higher, achievable operating frequency and sustained processing throughput. Moreover, independent multiplications can be distributed across multiple floating-

point units, while balanced adder trees help reduce the depth of dependent accumulation operations (compared with serial accumulations). Furthermore, precomputed parameters can be organized using memory banking or replication to i) support concurrent operand retrieval and ii) reduce memory-access contention (when multiple arithmetic units operate simultaneously). Thus, combining arithmetic pipelining, parameter-memory arrangement, and parallel processing benefits an efficient use of hardware resources and a reduction in execution latency.

REFERENCES FOR SUPPLEMENTAL MATERIAL

- [29] L. C. Félix-Herrán, D. Mehdi, J. D. J. Rodríguez-Ortiz, V. H. Benitez, R. A. Ramirez-Mendoza, and R. Soto, “Disturbance rejection in a one-half semiactive vehicle suspension by means of a fuzzy- H_∞ controller,” *Shock Vib.*, vol. 2019, no. 1, p. 4532635, 2019.
- [30] D. Simon, *Optimal State Estimation: Kalman, H_∞ , and Nonlinear Approaches*. USA: John Wiley & Sons, 2006.